

Module - 3

Clipping and Projections

- Window to viewport transformation. Cohen Sutherland Line clipping algorithm. Sutherland Hodgeman Polygon clipping algorithm. Three dimensional viewing pipeline. Projections- Parallel and Perspective projections. Visible surface detection algorithms- Depth buffer algorithm, Scan line algorithm.

1

Two-Dimensional Viewing

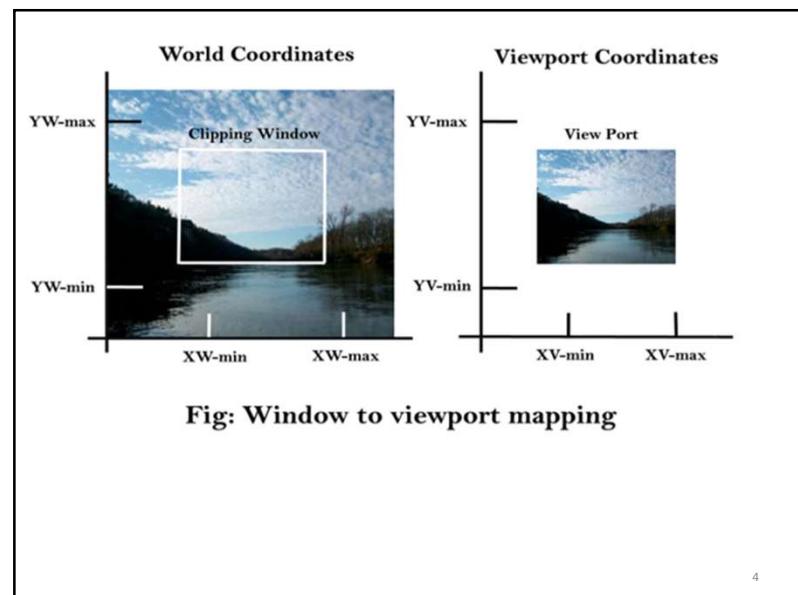
- A graphics package allows a user to specify **which part of a defined picture is to be displayed and where that part is to be placed on the display device.**
- Any convenient Cartesian coordinate system, referred to as the **world-coordinate reference frame**, can be used to define the picture.
- For a two-dimensional picture, a **view** is selected by specifying a subarea of the total picture area.
- A user can select a single area for display, or several areas could be selected for simultaneous display or for an animated panning sequence across a scene.
- The picture parts within the selected areas are then mapped onto specified areas of the device coordinates.
- When multiple view areas are selected, these areas can be placed in separate display locations, or some areas could be inserted into other, larger display areas.
- Transformations from **world to device coordinates** involve translation, rotation, and scaling as well as procedures for deleting those parts of the picture that are outside the limits of a selected display area.

2

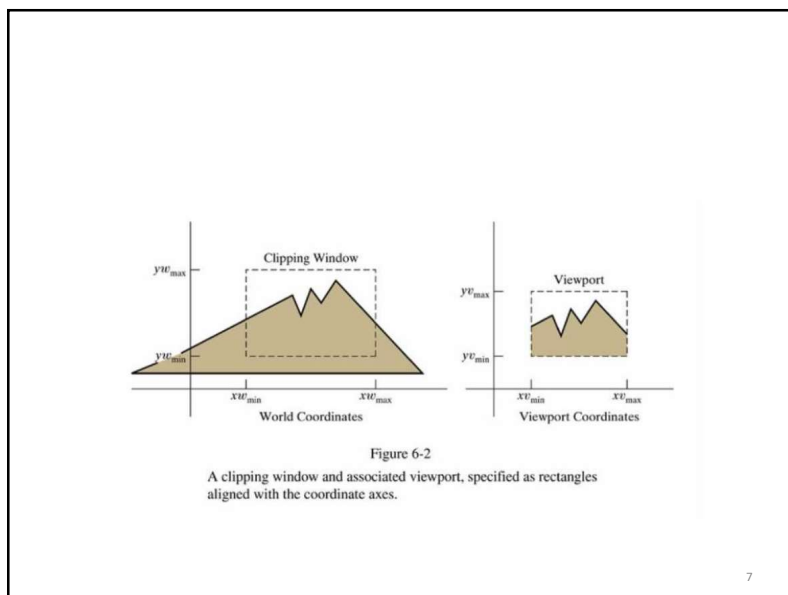
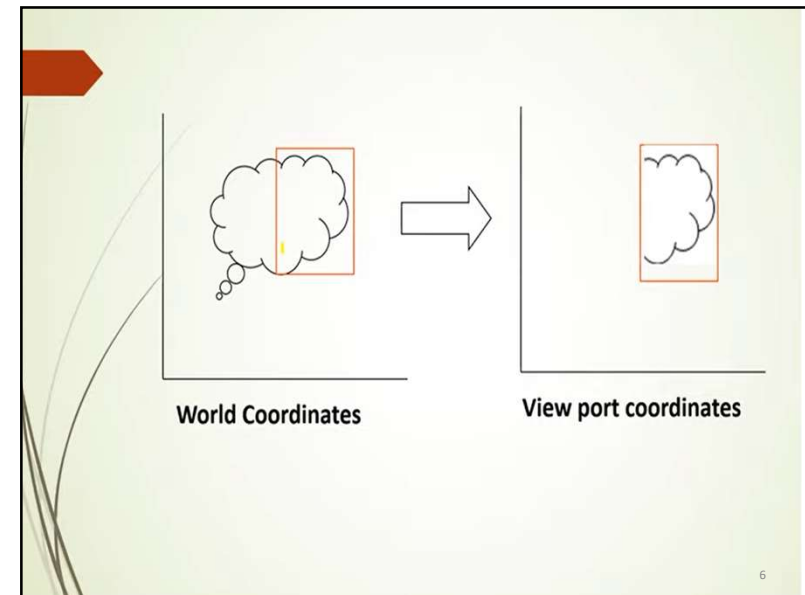
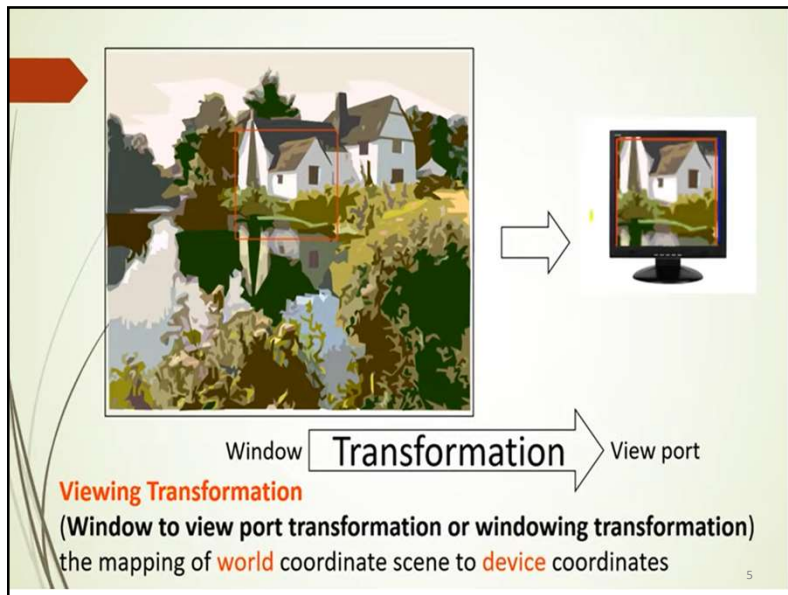
Window to viewport transformation

- Window (area where image is viewed in real world)**
 - A world-coordinate area selected for display.
 - The window defines what is to be viewed
- Viewport (area where image is viewed in o/p device)**
 - An area on a display device to which a window is mapped.
 - The **viewport** defines where it is to be displayed.
- Windows and viewports are **rectangles** in standard position, with the rectangle edges parallel to the coordinate axes.
- Viewing transformation:**
 - The mapping of a part of a world-coordinate scene to device coordinates.

3



4



1. Modeling Coordinates (Local/Object Coordinates)

- Defines positions **relative to an object's own coordinate system**.
- Used in 3D modeling or graphics to define an object before placing it in a larger scene.
Eg: A cube centered at the origin (0,0,0) with vertices like (1,1,1) and (-1,-1,-1) in its local space.

2. World Coordinates

- Defines positions in a **global reference system** for an entire scene.
- Objects are placed in the world using transformations like translation, rotation, and scaling.
Eg: A building model positioned at (50,100,0) in a game world.

3. Device Coordinates (Screen Coordinates)

- Defines positions **in terms of pixels on a display or output device**.
- The coordinate system of the physical screen where world coordinates are projected.
Eg: A point in world space is mapped to (300, 400) pixels on a 1920x1080 display.

4. Normalized Coordinates

- A coordinate system where values are scaled **within a fixed range**, typically [0,1] or [-1,1].
- Used in computer graphics, rendering, and UI design to make transformations device-independent.
- Ensures consistency across different screen sizes and resolutions.

Viewing Coordinates (Camera Coordinates)

- A coordinate system **relative to the camera or viewer's perspective** in 3D graphics.
- Also called **Eye Coordinates** or **View Space**.
- Used to transform objects from **world space** into a coordinate system where the camera is at the origin **(0,0,0)** and looking in a specific direction.
- **Transformation to Viewing Coordinates**
- To convert world coordinates to viewing coordinates, a **view transformation (camera transformation)** is applied:

1. Translation: Moves the camera to the origin.

2. Rotation: Aligns the camera's orientation.

9

Transformations Between Coordinate Systems

- To convert between these coordinate systems, various mathematical transformations are used:

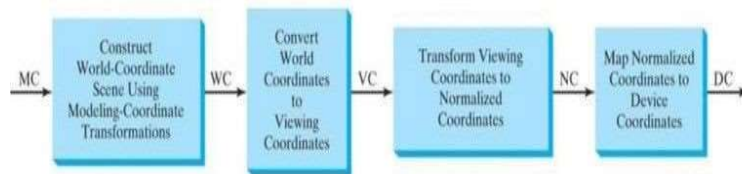
1. Model-to-World Transformation (Translation, Rotation, Scaling)

2. World-to-View Transformation (Camera View)

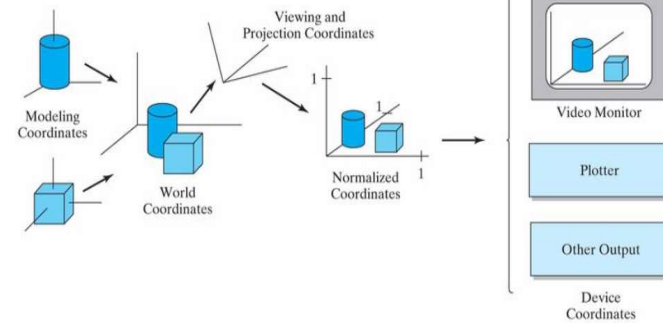
3. View-to-Device Transformation (Projection, Clipping, Mapping to screen coordinates)

10

Steps in 2D viewing pipeline



11



Transformation sequence from modeling coordinates to device coordinates

1. step: Object shapes can be individually defined in modeling-coordinate reference systems.
2. step: The shapes are positioned within the world-coordinate scene.
3. step: World-coordinate specifications are transformed through the viewing pipeline to viewing and projection coordinates and then to normalized coordinates.
4. step: Viewing coordinates translates to normalized coordinate
5. step: individual device drivers transfer the normalized-coordinate representation of the scene to the output devices for display.

12

- The viewing transformation is carried out in several steps.
- Construct the scene in world coordinates using the output primitives and attributes.
- Next, to obtain a particular orientation for the window, we can set up a two-dimensional viewing-coordinate system in the world-coordinate plane, and define a window in the viewing-coordinate system.
- The viewing coordinate reference frame is used to provide a method for setting up arbitrary orientations for rectangular windows.

13

- Once the viewing reference frame is established, we can transform descriptions in world coordinates to viewing coordinates.
- We then define a viewport in normalized coordinates (in the range from 0 to 1) and map the viewing-coordinate description of the scene to normalized coordinates.
- At the final step, all parts of the picture that are outside the viewport are clipped, and the contents of the viewport are transferred to device coordinates.

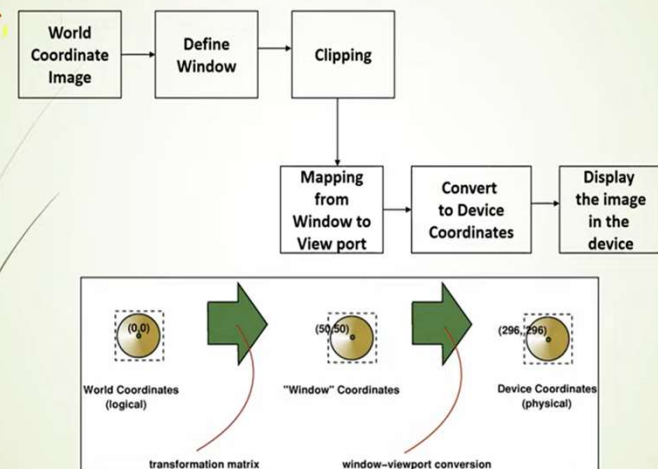
14

WINDOW-TO-VIEWPORT COORDINATE TRANSFORMATION

- Once object descriptions have been transferred to the viewing reference frame, we choose the window extents in viewing coordinates and select the viewport limits in normalized coordinates.
- Object descriptions are then transferred to normalized device coordinates.
- We do this using a transformation that maintains the same relative placement of objects in normalized space as they had in viewing coordinates.
- If a coordinate position is at the center of the viewing window, for instance, it will be displayed at the center of the viewport.

15

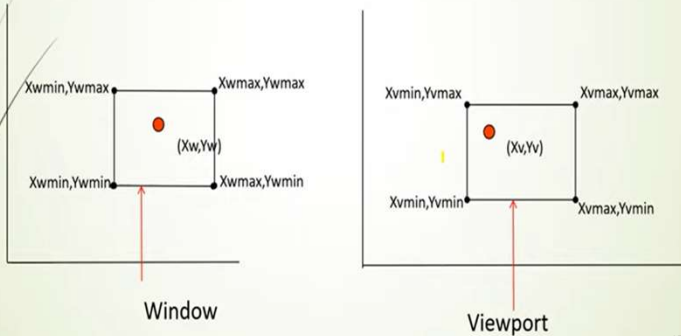
Sequence of Viewing Transformation



16

Window to view port transformation

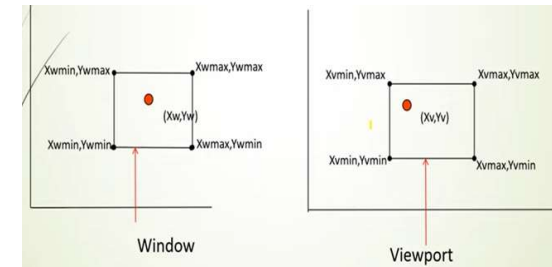
For every point (X_w, Y_w) in the window there is a **corresponding or equivalent or relative point** (X_v, Y_v) in view port



17

$$\text{Normalized Point on Window} = \left(\frac{X_w - X_{wmin}}{X_{wmax} - X_{wmin}}, \frac{Y_w - Y_{wmin}}{Y_{wmax} - Y_{wmin}} \right)$$

$$\text{Normalized Point on Viewport} = \left(\frac{X_v - X_{vmin}}{X_{vmax} - X_{vmin}}, \frac{Y_v - Y_{vmin}}{Y_{vmax} - Y_{vmin}} \right)$$



18

- In order to maintain the same relative placement of the point in the viewport as in the window, we require

$$\frac{X_w - X_{wmin}}{X_{wmax} - X_{wmin}} = \frac{X_v - X_{vmin}}{X_{vmax} - X_{vmin}} \quad \text{-----(1)}$$

$$\frac{Y_w - Y_{wmin}}{Y_{wmax} - Y_{wmin}} = \frac{Y_v - Y_{vmin}}{Y_{vmax} - Y_{vmin}} \quad \text{-----(2)}$$

$$\text{From (1) } X_v = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}} \times (X_w - X_{wmin}) + X_{vmin}$$

$$\text{From (2) } Y_v = \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}} \times (Y_w - Y_{wmin}) + Y_{vmin}$$

19

$$X_v = X_{vmin} + (X_w - X_{wmin}) S_x \quad Y_v = Y_{vmin} + (Y_w - Y_{wmin}) S_y$$

where s_x is the scaling factor of x coordinate and s_y is the scaling factor of y coordinate

$$S_x = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}}$$

$$S_y = \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}}$$

So window to viewport transformation involves scaling and translation

20

PROBLEM

- for window, $X_{wmin} = 20, X_{wmax} = 80, Y_{wmin} = 40, Y_{wmax} = 80$.
- for viewport, $X_{vmin} = 30, X_{vmax} = 60, Y_{vmin} = 40, Y_{vmax} = 60$.
- Now a point (X_w, Y_w) be $(30, 80)$ on the window. We have to calculate that point on the viewport i.e (X_v, Y_v) .

$$S_x = (60 - 30) / (80 - 20) = 30 / 60$$

$$S_y = (60 - 40) / (80 - 40) = 20 / 40$$

- So, now calculate the point on the viewport (X_v, Y_v) .

$$X_v = 30 + (30 - 20) * (30 / 60) = 35$$

$$Y_v = 40 + (80 - 40) * (20 / 40) = 60$$

So, the point on window $(X_w, Y_w) = (30, 80)$ will be $(X_v, Y_v) = (35, 60)$ on viewport.

21

Clipping

- Any procedure that identifies those portions of a picture that are either inside or outside of a specified region of space is referred to as a clipping algorithm or clipping.
- The region against which an object is to be clipped is called a clip window.
- Clipping is the process of **eliminating the portions outside the window**
- 2 types
 - wrt window
 - Clip while selecting the window
 - wrt viewport
 - Clip after saving the image to the view port

22

Clipping

- It is an inside-outside procedure
 - Define the window area
 - Discarding portions outside window is called clipping



Clip Window



After Clipping

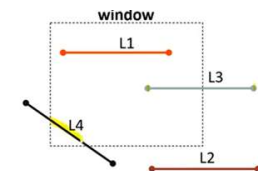
Before Clipping

23

Line Clipping

With respect to a window, the line can be

- perfectly inside (L1)
- perfectly outside (L2)
- partially inside (L3)
- endpoints outside, line segment inside (L4)

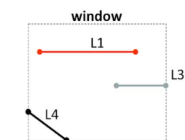


If the line
is perfectly inside (L1) , accept the line

is perfectly outside (L2) , reject the line

crosses window boundary , clip the line

After clipping



24

Cohen-Sutherland line clipping algorithm

- This is one of the oldest and most popular line-clipping procedures.
- The method speeds up the processing of line segments by performing initial tests that reduce the no. of intersections that must be calculated.
- Every **line endpoint** in a picture is **assigned** a **four-digit binary code**, called a **region code**, that identifies the **location of the point relative to the boundaries of the clipping rectangle**.

25

- Each bit position in the region code is used to indicate one of the four relative coordinate positions of the point with respect to the clip window: **to the left, right, top, or bottom**. By numbering the bit positions in the region code as 1 through 4 from right to left, the coordinate regions can be correlated with the bit positions as

bit 1: left
bit 2: right
bit 3: below
bit 4: above

1001	1000	1010
0001	0000 Window	0010
0101	0100	0110

A value of 1 in any bit position indicates that the point is in that relative position; otherwise, the bit position is set to 0

bit4	bit3	bit2	bit1
above	below	right	left

- If a point is within the clipping rectangle, the region code is **0000**. A point that is below and to the left of the rectangle has a region code of **0101**.
- Bit values in the region code are determined by comparing endpoint coordinate values (x, y) to the clip boundaries

- Bit 1 is set to 1 if $x < xw_{min}$.
- Bit 2 is set to 1 if $x > xw_{max}$
- Bit 3 is set to 1 if $y < yw_{min}$.
- Bit 4 is set to 1 if $y > yw_{max}$

27

Cohen Sutherland Line Clipping Algorithm

- Step 1 : Assign a region code for two endpoints of given line.
- Step 2 : If both endpoints have a region code 0000 then given line is completely inside (accept line)
- Step 3 : Else, perform the logical AND operation for both region codes.
- Step 3.1 : If the result is not 0000, then given line is completely outside. (reject line)
- Step 3.2 : Else line is partially inside. (need clipping)
- Step 3.2.1 : Choose an endpoint of the line that is outside the window
- Step 3.2.2 : Find the intersection point at the window boundary using equations below
- Intersections with a vertical boundary:
 $y = y1 + m(x - x1)$ where x is xw_{min} or xw_{max}
- Intersections with a horizontal boundary:
 $x = x1 + (y - y1)/m$ where y is yw_{min} or yw_{max}
- Step 3.2.3 : Replace endpoint with the intersection point and update region code.
- Step 3.2.4 : Repeat step 2 until we find a clipped line either trivially accepted or trivially rejected.
- Step 4 : Repeat step 1 for other lines .

28

*line L1

ENDPOINTS

0000

0000

-----AND

0000

Completely inside

*Line L2

ENDPOINTS

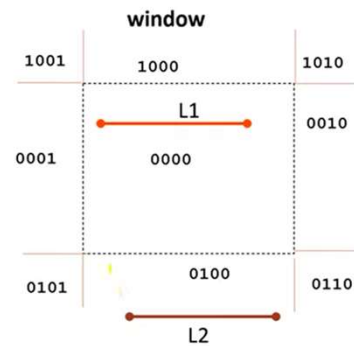
0100

0100

-----AND

0100

Completely outside



29

*line L3

ENDPOINTS

0000

0010

-----AND

0000

Partially inside .need clipping

Find intersection point

New ENDPOINTS

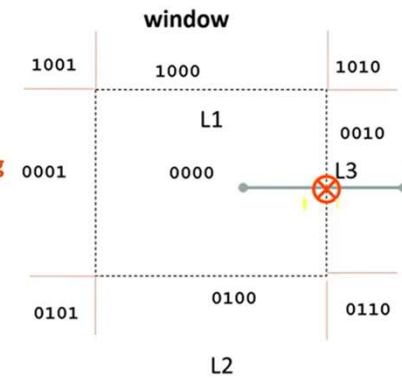
0000

0000

-----AND

0000

Completely Inside=>Accept



30

*line L4

ENDPOINTS

0001

0100

-----AND

0000

Partially inside .need clipping

Find intersection point

New ENDPOINTS

0001

0000

-----AND

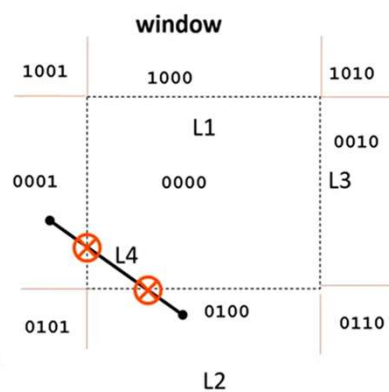
0000

Partially Inside. need clipping

Find intersection point

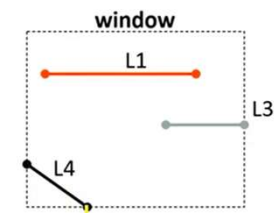
New endpoints

0000 & 0000 → 0000 → completely inside accept



31

Final clipped lines

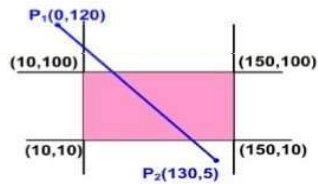


32

Contd..

- **Example:** Use the Cohen-Sutherland line clipping algorithm to clip line defined with end points $P_1(0,120)$ and $P_2(130,5)$ against a window defined by the following four points : $(10,10)$, $(10,100)$, $(150,10)$ and $(150,100)$

Obtain the endpoints of line P_1P_2 after cohen-sutherland clipping

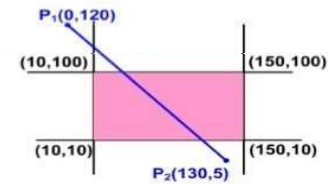


33

Contd..

- **Example:** Use the Cohen-Sutherland line clipping algorithm to clip line defined with end points $P_1(0,120)$ and $P_2(130,5)$ against a window defined by the following four points : $(10,10)$, $(10,100)$, $(150,10)$ and $(150,100)$

Obtain the endpoints of line P_1P_2 after cohen-sutherland clipping



bit4	bit3	bit2	bit1
above	below	right	left

34

Contd...

- **Solution:**

$$1. P_1=1001 \quad P_2=0100$$

$$2. \text{Both } (0000) \rightarrow \text{NO}$$

$$3. \text{And Operation}$$

$$1001$$

$$0100$$

$$\text{Result} \rightarrow 0000$$

$$3.1 \text{ not}(0000) \rightarrow \text{NO}$$

$$3.2 \quad 0000 \rightarrow \text{yes}$$

$$3.2.1 \text{ choose } P_1$$

$$3.2.2 \text{ Intersection with LEFT boundary}$$

$$m = (5-120)/(130-0) = -0.8846$$

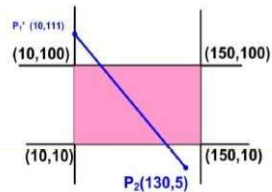
$$y = y_1 + m(x-x_1) \text{ where } x=10$$

$$y = 120 - 0.8846(10-0) = 111.15 \approx 111$$

$$P_1' = (10, 111)$$

$$3.2.3 \text{ Update region code } P_1' = 1000 \text{ (TOP)}$$

$$3.2.4 \text{ Repeat step 2.}$$



bit4	bit3	bit2	bit1
above	below	right	left

35

Contd...

$$1. P_1' = 1000 \quad P_2 = 0100$$

$$2. \text{Both } (0000) \rightarrow \text{NO}$$

$$\text{And Operation}$$

$$1000$$

$$0100$$

$$\text{Result} \rightarrow 0000$$

$$3.1 \text{ not}(0000) \rightarrow \text{NO}$$

$$3.2 \quad 0000 \rightarrow \text{yes}$$

$$3.2.1 \text{ choose } P_1'$$

$$3.2.2 \text{ Intersection with TOP boundary}$$

$$m = (5-120)/(130-0) = -0.8846$$

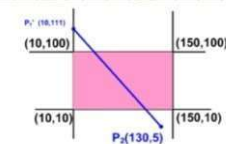
$$x = x_1 + (y-y_1)/m \text{ where } y=100$$

$$x = 10 + (100-111)/(-0.8846) = 22.44 \approx 22$$

$$P_1'' = (22, 100)$$

$$3.2.3 \text{ Update region code } P_1'' = 0000$$

$$3.2.4 \text{ Repeat step 2.}$$



36

Contd...

1. $P_1'' = 0000$ $P_2 = 0100$ 2. Both (0000) $\rightarrow$ NO

And Operation

0000

0100

Result $\rightarrow$ 00003.1 not(0000) $\rightarrow$ NO3.2 0000 $\rightarrow$ yes3.2.1 choose P_2 3.2.2 Intersection with **BOTTOM** boundary

$$m = (5-10)/(130-0) = -0.8846$$

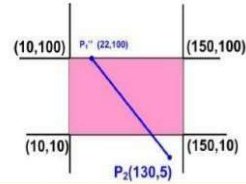
$$x = x_1 + (y - y_1)/m \quad \text{where } y = 10$$

$$x = 130 + (10-5)/(-0.8846) = 124.35 \approx 124$$

$$P_2' = (124, 10)$$

3.2.3 Update region code $P_2' = 0000$

3.2.4 Repeat step 2.



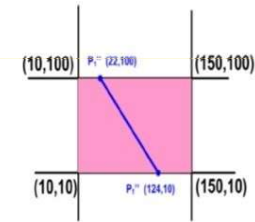
37

Contd...

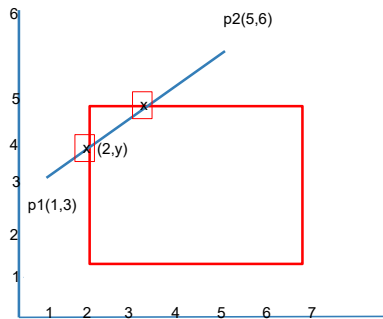
Both (0000) $\rightarrow$ YES

ACCEPT & DRAW

Thus endpoints after clipping

 $P_1'' = (22, 100)$ $P_2' = (124, 10)$ 

38

Example 2:Clipping window lower left(2,1) , upper right(7,5)
Line points (1,3),(5,6)

P1 region code:0001

P2 region code:1000

P1 AND P2

0001

1000

0 0 0 0

P1 intersecting Left edge

$$y = y_1 + m(x - x_1)$$

$$x = x_{wmin}$$

$$m = (6-3)/(5-1) = 3/4 = 0.75$$

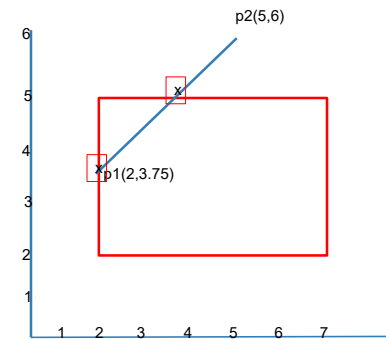
$$X = 2$$

$$y = 3 + 0.75(2-1) = 3.75$$

(x,y)=(2,3.75) --new end point p1

P1 New region code 0000
completely inside

39

Clipping window lower left(2,1) , upper right(7,5)
Line points (1,3),(5,6)

P1 region code:0000

P2 region code:1000

P1 AND P2

0000

1000

0 0 0 0

P2 intersecting above edge

$$x = x_1 + (y - y_1)/m$$

$$\text{where } y = y_{wmax}$$

$$m = (6-3)/(5-1) = 3/4 = 0.75$$

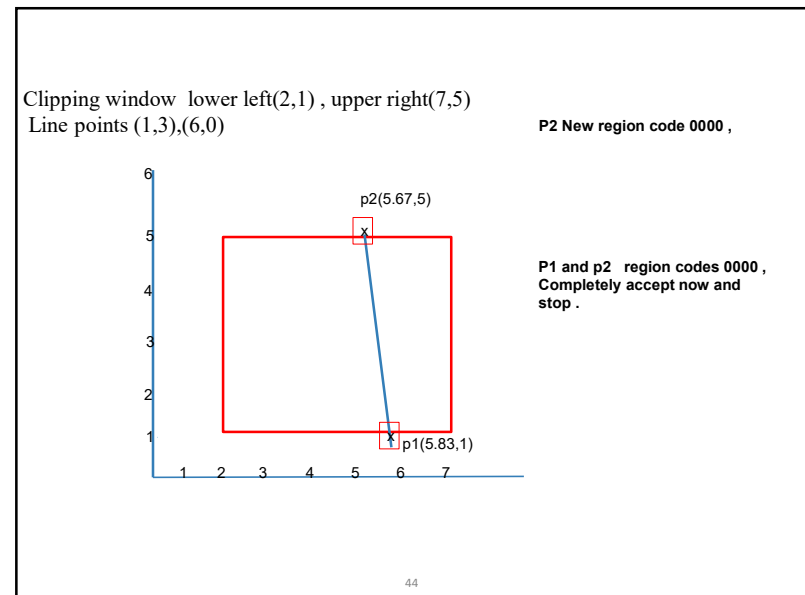
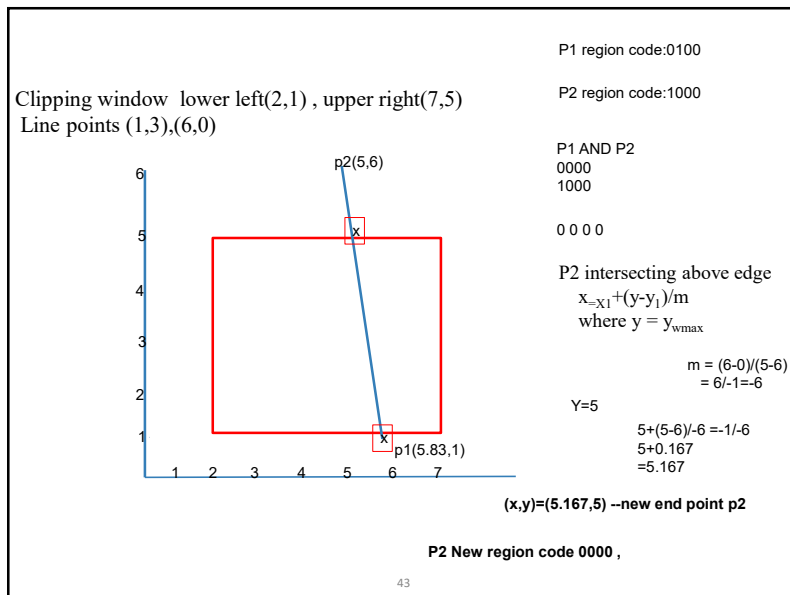
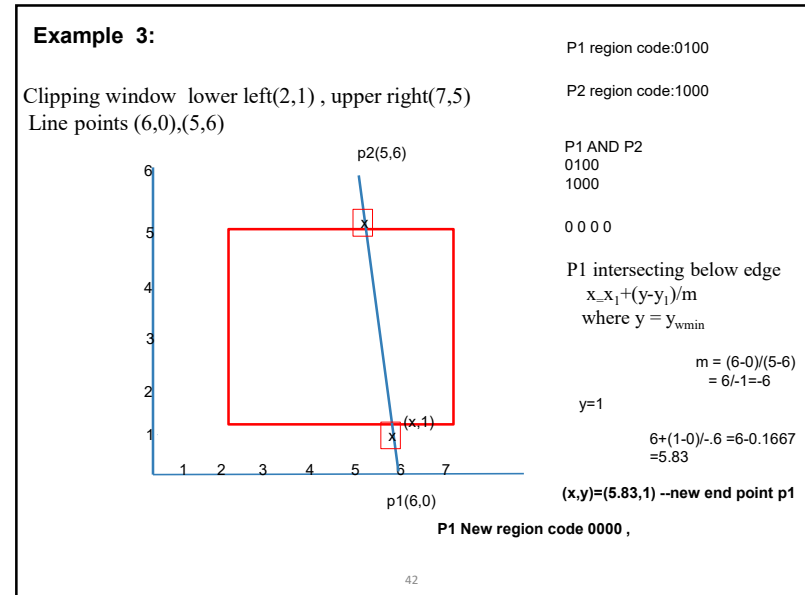
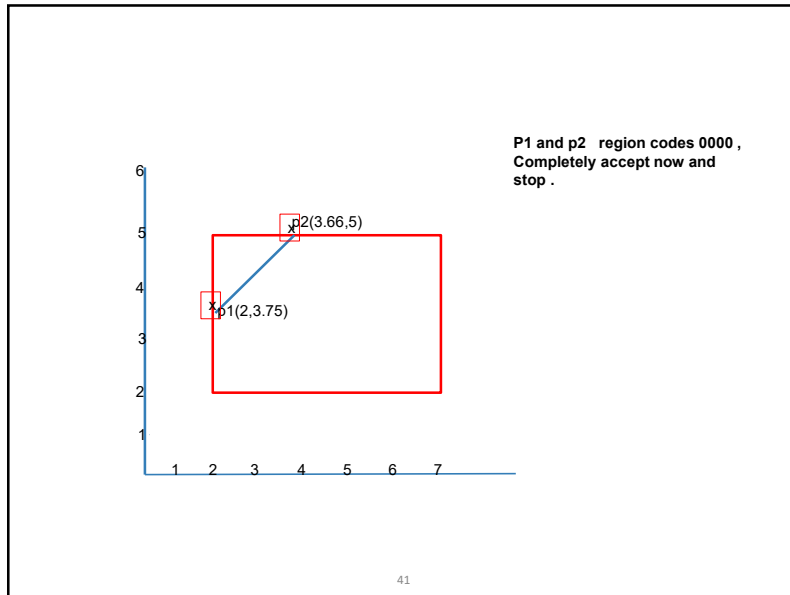
$$y = 5$$

$$x = 5 + (5-6) / .75 = 5 - (1.33) = 3.66$$

(x,y)=(3.66,5) --new end point p2

P2 New region code 0000

40



Sutherland Hodgeman Polygon clipping algorithm.

- It is performed by **processing the boundary of polygon against each window corner or edge.**
- Beginning with the initial set of polygon vertices, first clip the polygon against the **left** rectangle boundary to produce a new sequence of vertices.
- The new set of vertices are successively passed to a **right** boundary clipper, a **bottom** boundary clipper, and a **top** boundary clipper.
- At each step, a **new sequence of output vertices is generated and passed to the next window boundary clipper.**

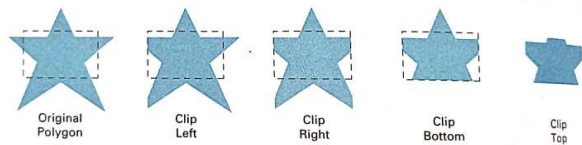


Figure 6-19
Clipping a polygon against successive window boundaries.

45

There are four possible cases when processing vertices in sequence around the perimeter of a polygon:

1. If the first vertex is outside window boundary and the second vertex is inside, both the intersection point of polygon edge with the window boundary and the second vertex are added to the output vertex list.
2. If both input vertices are inside the window boundary, only the second vertex is added to the output vertex list.
3. If the first vertex is inside the window boundary and the second vertex is outside the window boundary, only the edge intersection with the window boundary is added to output vertex first.
4. If both input vertices are outside the window boundary, nothing is added to the output list.

46

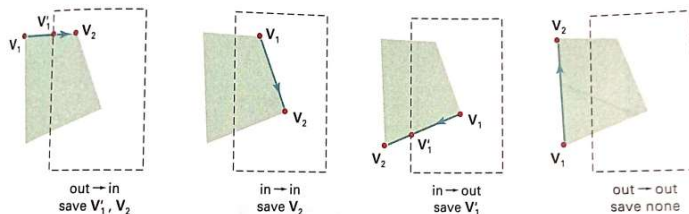


Figure 6-20
Successive processing of pairs of polygon vertices against the left window boundary.

47

Disadvantages of Sutherland Hodgeman Polygon clipping algorithm:

- This method requires a considerable amount of memory.
 - First all polygons are stored in original form. Then clipping against left edge is done and output is stored. Then clipping against right edge done, then top edge. Finally, the bottom edge is clipped. Results of all these operations are stored in memory- Wastage of memory for storing intermediate polygons.
- Convex polygons are properly clipped, but concave polygons may be displayed with extraneous lines.

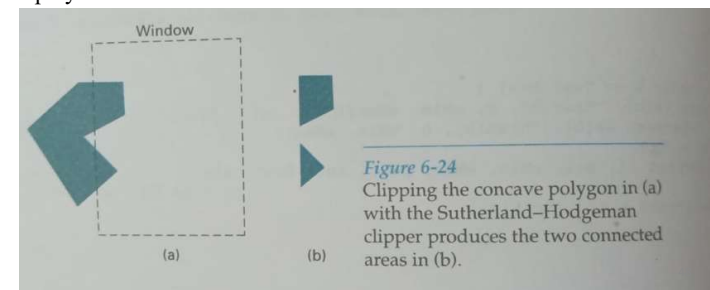
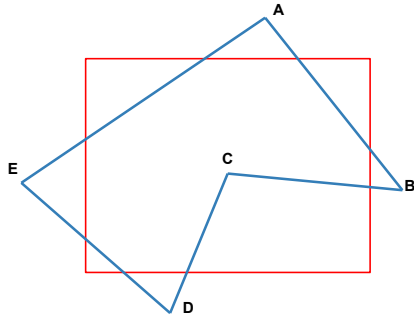


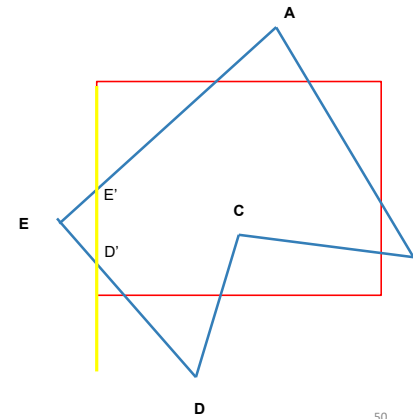
Figure 6-24
Clipping the concave polygon in (a) with the Sutherland-Hodgeman clipper produces the two connected areas in (b).

Clip the Polygon using Sutherland Hodgemen Algorithm:



49

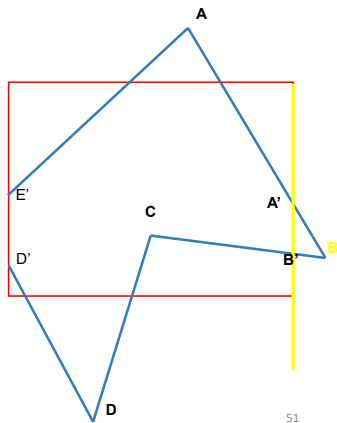
Left clip



VERTEX	CASE	OUTPUT
AB	in -> in	B
BC	in -> in	C
CD	in -> in	D
DE	in -> out	D'
EA	out -> in	E'A

50

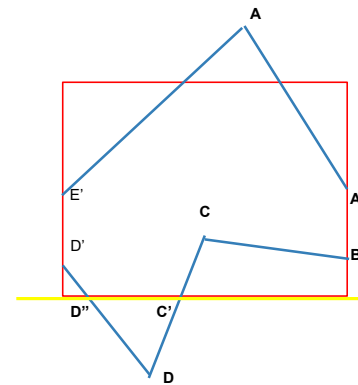
Right clip



51

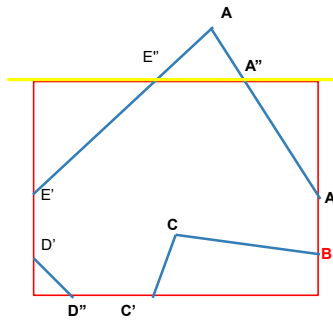
VERTEX	CASE	OUTPUT
AB	in -> out	A'
BC	out -> in	B'C
CD	in -> in	D
DD'	in -> in	D'
D'E'	in -> in	E'
E'A	in -> in	A

Bottom clip



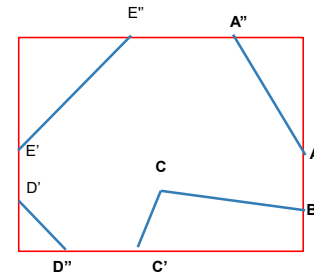
52

VERTEX	CASE	OUTPUT
AA'	in -> in	A'
A'B'	in -> in	B'
B'C	in -> in	C
CD	in -> out	C'
DD'	out -> in	D'' D'
D'E'	in -> in	E'
E'A	in -> in	A

Top clip

VERTEX	CASE	OUTPUT
AA'	out->in	A'' A'
A'B'	in->in	B'
B'C	in->in	C
CC'	in->in	C'
C'D''	in->in	D''
D''D'	in->in	D'
D'E'	in->in	E'
E'A	in->out	E''

53

Final Polygon

54

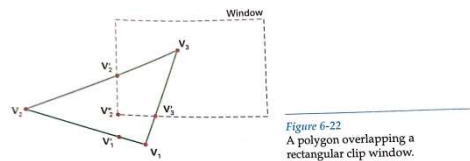
Example

Figure 6-22
A polygon overlapping a rectangular clip window.

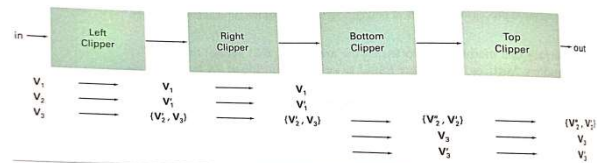


Figure 6-23
Processing the vertices of the polygon in Fig. 6-22 through a boundary-clipping pipeline. After all vertices are processed through the pipeline, the vertex list for the clipped polygon is $\{V_2, V_3, V_4, V_5\}$.

55